in isolation. Unit tests make it easier to focus on getting a particular class working and identifying what other services or components it needs. Unit tests also make it easier to detect and isolate errors or regressions. You'll typically write many small unit tests in order to get an acceptance criterion to pass (see figure 2.8).

You could use a conventional unit-testing framework such as JUnit for these unit tests. But because this book is about BDD, you'll use a more BDD-specific unit-testing tool called Spock. Spock is a lightweight and expressive BDD-style testing library for Java and Groovy applications that you'll use extensively later on in this book.

Groovy is a dynamic language built on top of Java that incorporates many features from languages like Ruby and Python. It's a very expressive language that works well for checking outcomes in a clean and concise manner.

Writing unit tests in Spock

In Spock, you write unit tests in the form of "specifications," using a very readable "given ... when ... then" structure similar to that used in the JBehave scenarios. For example, if you wanted to test a `Calculator` class in Spock, you could write something like this:

```
def "should calculate the sum of two numbers"() {
  given: "two numbers"
    int a = 1
    int b = 2
  when: "we add the numbers together"
    def calculator = new Calculator()
    int sum = calculator.add(a,b)
  then: "the result should be the sum of the two numbers"
    sum == 3
```

Defining a specification

The given clause

The when clause

The then clause

The equivalent of "assert sum == 3"

Many Java developers like to write unit tests using Spock because it lets them write concise and descriptive tests with less boilerplate code than would be needed using Java.

You'll start off by essentially testing the acceptance criteria described in the JBehave scenario in listing 2.1, but at a unit-testing level. In Spock, you could create a Groovy class called `WhenCalculatingArrivalTimes.groovy` in an appropriate package in the src/test/groovy directory.[13] This class might look something like the following:

```
package com.bddinaction.chapter2.services

import org.joda.time.LocalTime
import spock.lang.Specification

class WhenCalculatingArrivalTimes extends Specification {

    ItineraryService itineraryService;
```

Spock tests must extend the Specification class

[13] In the sample solution, we put this class in the com.bddinaction.chapter2.services package.